Database Design — Quick Reference Card

database-design-quick-reference.txt

```
================================================================================
 DATABASE DESIGN - QUICK REFERENCE CARD
 One page-set you keep open beside every other file. All the rules, drawn.
================================================================================

 This file invents nothing new. It is a dense, scannable summary of everything
 the other files teach, so that once you have READ them you have a single place
 to glance at while you design. Every rule here has a tiny drawn example, because
 a rule you cannot picture is a rule you will forget.

 WHERE THINGS LIVE (so you know which file to open for the long version):
   - the METHOD and the WHY ......... database-design-primer.txt
   - the BRIDGE / confusions ........ database-design-companion-lesson.txt
   - the PICTURES (tables+diagrams) . database-design-revised-lesson.md + -samples.md
   - the HANDS-ON (real SQL) ........ database-design-practice-lesson.txt
   - the cross-AI VERDICT ........... database-design-AI-pieces-review.txt
   - the finished REAL design ....... the student-panel project (read-only)

 Signpost legend (same four you already know):
   >> CONTEMPLATE   stop and think.      KEY IDEA   the one sentence to keep.
   TRY IT          a tiny paper task.    >> PITFALL  a named mistake to dodge.

 KEY IDEA: design lives on PAPER; a database is just a good paper design made
 real and filled with rows. Get the paper right and the SQL is short and obvious.

--------------------------------------------------------------------------------
 1 - THE DESIGN METHOD (the merged routine - do these in order)
--------------------------------------------------------------------------------

 This is the one routine that unifies the primer's Part 7 list and the other-AI
 "Lesson 15" list: it keeps the explicit "Draw it" step AND makes "questions/
 actions" and "delete rules" their own steps so you cannot skip them.

   1  PURPOSE ........ one sentence: what is this database FOR?
   2  QUESTIONS ...... list the questions it must answer + actions it must support.
   3  ENTITIES ....... the nouns worth keeping facts about  -> candidate tables.
   4  ATTRIBUTES ..... the facts about each entity           -> candidate columns.
   5  KEYS ........... pick a primary key per entity (usually a surrogate id).
   6  RELATIONSHIPS .. say each link aloud as a sentence; note cardinality.
   7  HARD PATTERNS .. resolve M:N, link-with-facts, ternary, 1:1, self-ref.
   8  PLACE FKs ...... put each foreign key on the MANY side.
   9  DELETE RULES ... choose RESTRICT / CASCADE / SET NULL for each FK.
  10  DRAW IT ........ a crow's-foot diagram of the whole thing.
  11  NORMALISE ...... check 1NF, 2NF, 3NF; fix or consciously denormalise.
  12  MAP TO TABLES .. write the final table list: columns, keys, FKs, NOT NULL.
  13  TEST WITH ROWS . invent sample rows; answer your Step-2 questions against
                       them. If a question is awkward, the design is wrong - loop.

 KEY IDEA: steps 1-2 and step 13 are the same thing at both ends - you start
 from what it must do, and you finish by proving it does it.

--------------------------------------------------------------------------------
 2 - THE ROSETTA STONE: ONE IDEA, THREE COSTUMES
--------------------------------------------------------------------------------

 The same thing has three names depending on where you are standing. Learn the
```

row of this table and the whole subject stops feeling like three subjects.

ON PAPER (ER)	IN THE DATABASE	WHAT IT IS
entity type	table	the KIND of thing
entity instance	row	one actual thing
attribute	column	one fact kept about each
identifier	primary key	the handle that names a row
relationship	foreign key / table	how two kinds connect
the whole design	schema	tables + columns + rules

Same single idea - "we keep students, each with a name and a join date" - in all three costumes:

(a) PAPER: student = (student_id [key], full_name, joined_on)

(b) GRID:

student_id	full_name	joined_on
1	Sara Ahmadi	2026-09-01
2	Omid Karimi	2026-09-03

(c) SQL:

```
CREATE TABLE student (
  student_id  INTEGER PRIMARY KEY,
  full_name   TEXT NOT NULL,
  joined_on   DATE
);
```

The four SQL verbs you need for a long time:
CREATE TABLE -> build the design INSERT -> add a row
SELECT -> ask a question UPDATE / DELETE -> change / remove rows

---

3 - ENTITY OR ATTRIBUTE? (own table, or just a column?)

THE TEST: do you keep facts ABOUT this thing, or will many rows SHARE it?
- yes to either -> it is an ENTITY (own table), pointed at by a foreign key.
- no, it is one plain value for this one row -> it is an ATTRIBUTE (a column).

"Colour" is a COLUMN in a small shop (just a word on the product) but a TABLE in a paint factory (colours have codes, suppliers, recipes - facts about them). Purpose decides. Same word, different answer.

ATTRIBUTE flavours to recognise:
atomic (one value) vs composite (full_name = first + last - split if you ever query the parts); single-valued vs multi-valued (many phones -> own table, never phone1/phone2/phone3); stored vs derived (store birth_date, SHOW age).

---

4 - DEGREE, CARDINALITY, PARTICIPATION (the three dials of a relationship)

DEGREE = how many entity types the relationship touches:
unary (a thing links to its own kind: employee -> manager),
binary (two kinds: loan links member + copy) <- almost everything,
ternary (three kinds at once: doctor + drug + patient = prescription).

```
CARDINALITY = how many on each side:
  1:1  one-to-one   (person <-> passport)
  1:N  one-to-many  (member -> many loans)        <- the common one
  M:N  many-to-many (student <-> course)          <- needs a bridge table

PARTICIPATION = is a side optional or mandatory (the min)?
  optional  (zero allowed): a brand-new member may have 0 loans.
  mandatory (at least one):  a loan MUST have exactly one member.

>> CONTEMPLATE: say every link as a sentence BOTH ways before you draw it:
"one member has zero-or-many loans; each loan belongs to exactly one member."
The two halves give you both ends of the crow's-foot line.

--------------------------------------------------------------------------------
5 - CROW'S-FOOT CHEAT-SHEET (the little marks on the line)
--------------------------------------------------------------------------------

A line joins two tables. Near EACH end are up to two marks. Read them OUTSIDE-IN
toward the table: the FAR mark is the MINIMUM, the NEAR mark is the MAXIMUM.

    |   bar          -> "one"      (max: exactly one)
    <   crow's foot   -> "many"     (max: many)
    o   circle        -> "zero"     (min: optional)
    |   (as min)      -> "one"      (min: mandatory)

  +--------+----------------------+-----------------------------------+
  | symbol | means                | plain English                     |
  +--------+----------------------+-----------------------------------+
  |  ||    | one and only one     | min 1, max 1  (mandatory, single) |
  |  o|    | zero or one          | min 0, max 1  (optional, single)  |
  |  |<    | one or many          | min 1, max many                   |
  |  o<    | zero or many         | min 0, max many                   |
  +--------+----------------------+-----------------------------------+

THE MIRROR: the same mark faces the line from whichever side it is on. "Zero or
many" is  o<  on a left end and  >o  on a right end - ONE idea, two reflections.
Do not read ">o" and "o<" as different things.

TINY DRAWN EXAMPLES (max/min shown next to each box):

  one-to-many (a member has many loans; a loan has one member):

      +--------+                          +-------+
      | MEMBER |||--------------------o<| LOAN  |
      +--------+  one          zero/many +-------+

  many-to-many ALWAYS becomes two one-to-manys through a bridge:

      +---------+          +-----------+          +--------+
      | STUDENT |||----o<| ENROLMENT |>o----||| COURSE |
      +---------+          +-----------+          +--------+

WORKED READ (the library line):
      MEMBER ||----o<  LOAN  >o----|| COPY
  Member side: "||" at Member = one Member; "o<" at Loan = zero-or-many Loans
    -> one member has zero-or-many loans; each loan has exactly one member.
  Copy side:   ">o" at Loan = zero-or-many Loans; "||" at Copy = one Copy
    -> one copy has zero-or-many loans; each loan is of exactly one copy.

>> PITFALL: the crow's foot (many) sits on the end that HOLDS THE FOREIGN KEY.
"Many loans per member" => the LOAN table carries member_id, not the reverse.
```

```
--------------------------------------------------------------------------------
 6 - KEYS
--------------------------------------------------------------------------------

   CANDIDATE key : any column(s) that could uniquely name a row.
   PRIMARY key   : the candidate you chose as THE handle. One per table.
   NATURAL key   : a real-world value (email, ISBN, national id).
   SURROGATE key : an id invented by the system to be a stable handle.
   COMPOSITE key : a key made of two or more columns together.
   FOREIGN key   : a column that holds another table's primary key (the pointer).

 NATURAL vs SURROGATE: natural values change and repeat (people change email,
 names collide). A surrogate never changes and never repeats. MODERN DEFAULT:
 give every table a surrogate primary key AND keep the useful natural value under
 a UNIQUE rule so it still cannot duplicate.

 INT vs UUID - BOTH are surrogate keys, not a contradiction:
   +-------------------------+-----------------------------------------------+
   | INTEGER auto-number     | UUID (long random id)                         |
   | 1, 2, 3 ...             | 7f3a2b1c-...-9c2b                              |
   | small, easy to read,    | unique across many machines; hides "how many" |
   | great for LEARNING      | and "what order"; used by student-panel       |
   +-------------------------+-----------------------------------------------+
   student-panel = UUID primary key + a unique human code (student_code).
   The practice lesson uses INT ids ON PURPOSE (easier to type and see).

 REFERENTIAL INTEGRITY: a foreign key must point at a row that EXISTS. A pointer
 to nothing is an ORPHAN, and the database refuses to create one. That refusal is
 a feature - it is the database protecting your design from lies.

   +--------+        +------------------------------+
   | MEMBER |        | LOAN                         |
   +--------+        +----+-----------+-------------+
   | id | . |        | id | member_id | copy_id ... |
   | 1  |..|  <------| 10 |    1      |   ...       |  OK: member 1 exists
   | 2  |..|  <------| 11 |    2      |   ...       |  OK: member 2 exists
   +--------+   X----| 12 |    9      |   ...       |  REFUSED: no member 9
                     +----+-----------+-------------+

 WHEN A BRIDGE/TERNARY GETS ITS OWN surrogate id: decide whether the natural
 combination should also be UNIQUE. (order_line: UNIQUE(order_id, product_id)
 unless a repeat is truly allowed. Say the decision out loud and write it down.)

--------------------------------------------------------------------------------
 7 - THE FIVE HARD PATTERNS AND "THE ONE MOVE"
--------------------------------------------------------------------------------

 THE ONE MOVE: when a relationship is messy, put a TABLE IN THE MIDDLE. Three
 different-looking problems share this one solution:

 (1) MANY-TO-MANY  (student <-> course): a line cannot hold it. Add a BRIDGE.
       STUDENT ||--o< ENROLMENT >o--|| COURSE
       enrolment = ( student_id, course_id )   [PK = the pair]

 (2) A LINK WITH ITS OWN FACTS (the enrolment has a grade and a date): same
     bridge, now carrying the facts.
       enrolment = ( student_id, course_id, grade, enrolled_on )

 (3) A TERNARY (three-way) FACT (doctor + drug + patient = prescription): a table
     in the middle holding all three pointers.
```

```
      prescription = ( prescription_id, doctor_id, drug_id, patient_id, dose,
                       prescribed_on )   [decide: UNIQUE(doctor,drug,patient)?]
```

The other two patterns:

(4) ONE-TO-ONE (person <-> passport): usually MERGE into one table. If you keep them apart (optional half, or security), put the FK on the NEVER-EMPTY side with UNIQUE + NOT NULL.

```
  PERSON ||--o| PASSPORT    (passport.person_id UNIQUE NOT NULL)
```

(5) SELF-REFERENCE / TREE (a group has a parent group; an employee has a manager): one table with a parent_id FK pointing back at itself; the root has parent_id = NULL. Do NOT make parent/grandparent columns.

```
  +----------------------------+
  | GROUP                      |
  | id | name     | parent_id  |
  | 1  | School   |   NULL     |  <- root
  | 2  | Grade 7  |    1       |
  | 3  | Class 7A |    2       |
  +----------------------------+
```

PICK THE KEY STYLE FOR A BRIDGE (the C3 guidance):

- use the PAIR as the key when the pairing is naturally unique and nothing else needs to point at the bridge row;
- use a SURROGATE id when something else must reference the bridge row - and then keep a UNIQUE on the natural pair.

KEY IDEA: if a relationship has its own attributes, OR joins more than two things, OR is many-to-many, it wants to BE a table, not a line.

---

8 - ON DELETE RULES (what happens to the children when the parent goes)

Say each one as a plain sentence about YOUR data:

```
  ON DELETE RESTRICT  -> "you may NOT delete a member who still has loans."
                         (the safe default; blocks the delete)
  ON DELETE CASCADE   -> "delete a member and their loans vanish too."
                         (use only when the children are meaningless alone)
  ON DELETE SET NULL  -> "delete a category and its products become
                         uncategorised (category_id goes NULL)."
                         (needs the FK column to allow NULL)
```

>> PITFALL: CASCADE is convenient and dangerous. One delete can quietly erase a whole subtree. Choose it on purpose, never by habit.

---

9 - NORMALISATION (1NF, 2NF, 3NF) + THE THREE ANOMALIES

Mnemonic for 3NF: every non-key fact depends on
"THE KEY, THE WHOLE KEY, AND NOTHING BUT THE KEY."

```
  the key          -> 1NF (there IS a key; one value per cell)
  the whole key     -> 2NF (not just part of a composite key)
  nothing but the key -> 3NF (not on another non-key column)
```

1NF - one value per cell; no repeating groups.

```
  BAD:  student( id, name, subjects="math, science, art" )   <- list in a cell
  BAD:  student( id, name, phone1, phone2, phone3 )           <- repeating group
  FIX:  move the repeating thing to its OWN table with a FK back.
```

```
        student_subject( student_id, subject )   student_phone( student_id, phone )

 2NF - no non-key column depends on only PART of a composite key.
   BAD:  enrolment( student_id, course_id, course_title )
         (course_title depends on course_id alone, not the pair)
   FIX:  course_title belongs in COURSE, keyed by course_id.

 3NF - no non-key column depends on another non-key column (transitive).
   BAD:  student( id, name, group_id, group_name )
         (group_name depends on group_id, not on the student)
   FIX:  group_name belongs in GROUP, keyed by group_id.

 THE THREE ANOMALIES these prevent (why we bother):
   UPDATE: a repeated fact (group_name on every student) must be changed in many
           rows at once; miss one and the data now disagrees with itself.
   INSERT: you cannot record a new group until a student joins it (nowhere to put
           it) - the fact is held hostage by an unrelated row.
   DELETE: removing the last student in a group also erases the group's existence
           - you lose a fact you did not mean to lose.

 DENORMALISE only on purpose: copy a fact back for speed when you have MEASURED a
 real problem, and write down that you did it and why.

-------------------------------------------------------------------------------
 10 - DERIVE DON'T STORE - AND THE FROZEN-FACT EXCEPTION
-------------------------------------------------------------------------------

 Can a value be computed from other values? Then usually do NOT store it; compute
 it when asked (store birth_date, SHOW age; store line items, ADD UP the total).
 A stored derived value drifts out of step with the thing it came from.

 THE EXCEPTION - a FROZEN FACT: a value captured ON PURPOSE at a moment in time,
 which must never change afterward, is NOT derived - keep it.
   unit_price_at_sale on an order line: the product's price today is irrelevant;
   the order must remember what the customer actually paid.
   student-panel parallel: a recorded SCORE is a frozen fact of its period - once
   a period is scored, that number stands even if the parameter is later changed.

-------------------------------------------------------------------------------
 11 - DESIGN-JUDGEMENT QUICK CALLS
-------------------------------------------------------------------------------

   NULL ............ "no value here." Decide per column if blank is legal and
                     what it MEANS (loan.returned_on NULL = "not returned yet").
                     Forbid it (NOT NULL) where blank would be nonsense.
   EITHER/OR ....... one row points at A or at B but not both: two nullable FKs,
                     exactly one filled. (student-panel: an assignment targets a
                     group OR a single student.)
   SOFT vs HARD .... HARD delete erases the row. SOFT delete keeps it, marked
                     (archived_at / is_active), so attached history survives.
                     student-panel SOFT-deletes; the practice lesson HARD-deletes
                     a throwaway DB so you can SEE integrity refuse an orphan.
   RESERVED WORDS .. do not name a table order / group / user. Use orders /
                     groups / app_user. (Postgres - which student-panel uses -
                     treats all three as reserved.)
   JSON vs COLUMNS . if you ever filter, sort, or count by a value, make it a
                     real COLUMN. Keep JSON only for a genuine opaque blob.
   NAMING .......... singular or plural, but be consistent; lower_snake_case;
                     name the FK after what it points at (member_id).

-------------------------------------------------------------------------------
```

12 - STUDENT-PANEL AT A GLANCE (the real design, at the principle level)

This is the finished thing we measure our understanding against. We READ it; we never change it. You build your own version separately. Principles on show:

- UUID primary keys + a unique human code (student_code) ..... Section 6.
- a nested GROUPS tree and a nested PARAMETERS tree (parent_id, NULL at the root, cycles prevented) ..... pattern (5).
- a central SCORES / EVALUATIONS table that bridges a student and a parameter for a period and carries the score (a FROZEN fact) ..... the one move + 10.
- the either/or NULL pattern for assignments (group OR student) .. Section 11.
- SOFT delete / archive (archived_at) instead of erasing ..... Section 11.
- roles built from fine-grained permission keys; periods; an audit log.

KEY IDEA: everything in student-panel is one of the rules on this card, applied. Nothing in it is magic; it is this method, finished.

13 - CONSOLIDATED GLOSSARY

anomaly ..... a data problem (update/insert/delete) caused by a repeated fact; normalisation removes the cause.
attribute ..... a fact kept about an entity; becomes a column.
bridge table ... a table in the middle that resolves a many-to-many.
candidate key .. any column(s) that could uniquely identify a row.
cardinality .... how many on each side of a relationship (1:1, 1:N, M:N).
composite key .. a key made of more than one column.
degree ..... how many entity types a relationship joins (unary/binary/ternary).
denormalise .... deliberately store a redundant fact for speed, documented.
derived value .. a value computable from others; usually not stored.
entity ..... a kind of thing worth keeping facts about; becomes a table.
foreign key .... a column holding another table's primary key (the pointer).
frozen fact .... a value captured on purpose at a moment, kept forever.
instance ..... one actual thing; a row.
natural key .... a real-world identifier (email, ISBN).
normalisation .. organising columns so each fact lives in exactly one place.
NULL ..... "no value here"; legality and meaning decided per column.
orphan ..... a foreign key pointing at a row that does not exist (refused).
participation .. whether a side is optional (min 0) or mandatory (min 1).
primary key .... the chosen handle that names each row; one per table.
referential integrity .. the rule that every foreign key points at a real row.
relationship ... how two kinds of thing connect; a line or a middle table.
schema ..... the whole blueprint: tables + columns + rules.
self-reference . a table whose foreign key points back at itself (a tree).
soft delete .... mark a row as gone (archived_at) instead of erasing it.
surrogate key .. a system-invented id (INTEGER or UUID); never changes.
1NF ..... one value per cell; no repeating groups.
2NF ..... no non-key column depends on only part of a composite key.
3NF ..... no non-key column depends on another non-key column.

- end of quick reference -